

Cloud and Edge Computing

Compte rendu des TP



2025-2026

Jean-Philippe Loubejac Combalbert
Célian Hilal Hamdan

Enseignant : Sami Yanguï

Introduction

Les séances de TP effectuées dans le cadre du cours "Cloud and Edge Computing" visent à approfondir la compréhension des technologies de virtualisation et d'orchestration de services dans des environnements cloud et edge computing. À travers les TPs sur les hyperviseurs, VirtualBox, Docker et OpenStack, l'objectif est d'explorer les principes de la virtualisation, de la gestion de machines virtuelles et du déploiement d'infrastructures cloud. Le dernier TP introduit la mise en œuvre d'un cluster Kubernetes pour illustrer la gestion de services à faible latence dans un contexte d'edge computing, appliqué au cas des véhicules autonomes. Ensemble, ces TPs offrent une vision complète du cycle de vie des services virtualisés, de leur création à leur orchestration.

TP 1 : Introduction aux Hyperviseurs Cloud

1. Similitudes et différences entre les principaux hôtes de virtualisation (VM et CT)

Le schéma représente deux conceptions de hôtes virtuels : à gauche trois machines virtuelles avec un hyperviseur et à droite deux conteneurs.

Chaque VM embarque un système d'exploitation invité (Guest OS) au-dessus de l'hyperviseur de type 2, en plus de l'OS hôte. Chaque application repose donc sur son OS, ses bibliothèques et ses dépendances.

Les conteneurs partagent le même noyau de l'OS hôte et isolent uniquement les bibliothèques et dépendances nécessaires à l'application. Il n'y a pas d'OS invité.

	VM	CT
Coût de virtualisation	+ : Coût plus élevé dû à la présence d'un hyperviseur et d'un OS. Coûteuses en termes d'infrastructure.	- : Coût moins important (pas d'hyperviseur), meilleur temps de démarrage. Permet de tester rapidement plusieurs environnements. Possibilité de faire tourner plus d'instances sur un même serveur.
Utilisation CPU, mémoire et réseau	+ : Usage plus important de CPU et de mémoire (overhead) dû à la présence d'un OS supplémentaire. Mais plus facile (pour un admin) à allouer finement des ressources.	- : Les conteneurs ne nécessitent pas de système d'exploitation distinct pour chaque instance, utilisation réduite des ressources (CPU et RAM). Pour un admin, nécessite une gestion plus compliquée pour éviter la concurrence.
Security for the application	+ : Isolation de la mémoire entre les machines virtuelles. Une machine virtuelle exploitée sera isolée et ne pourra pas contaminer ses voisines en cas d'une menace de sécurité.	- : Isolation partielle car le même OS est utilisé. Si le noyau est compromis, tous les conteneurs sont vulnérables.
Performances	- : Temps de démarrage plus long car elles doivent configurer leur propre système d'exploitation. Overhead supplémentaire qui peut pénaliser certaines applications temps-réel (impact sur le temps de réponse).	+ : Démarrage rapide (quasi-instantané). Faible latence.

Outils pour l'intégration continue	-: Moins bonne prise en charge de l'intégration continue due au long temps de démarrage et l'utilisation accrue de ressources (Les images sont lourdes à créer, transporter et lancer. Cela ralentit fortement les pipelines d'intégration continue)	++: Meilleur pour la prise en charge de l'intégration continue (CI/CD) (certains le prennent en charge nativement). Un développeur peut enchaîner build, test et déploiement de manière fluide. Les conteneurs permettent également une reprise rapide en cas de panne, car il est possible de démarrer un nouveau conteneur avec la même configuration.
---	---	---

Pour un développeur, les conteneurs présentent un avantage majeur : ils sont faciles à mettre en place, rapides à lancer, légers, faciles à intégrer dans un pipeline CI/CD et permettent un usage de ressource plus faible.

Un administrateur serait quant à lui plus intéressé par une sécurité accrue, une prise de Snapshot simplifiée et par le déploiement d'une topologie réseau. Malgré une configuration plus compliquée, une machine virtuelle reste plus adaptée puisqu'elle présente l'avantage de séparer ses processus.

Ainsi, les **conteneurs** sont à privilégier dans ces cas d'utilisation :

- Applications de microservices
- Modernisation des applications existantes
- Intégration et livraison continues (CI/CD)

Les **machines virtuelles** sont à privilégier dans ces cas d'utilisation :

- Applications sensibles à la sécurité
- Exécution d'un système d'exploitation au sein d'un autre système
- Charges de travail gourmandes en ressources

2. Similitudes et différences entre les types de CT existants

	Linux CT	Docker CT
Isolation et ressources	Isolation des ressources (notamment processeur, mémoire et accès E/S). Moins bonne isolation applicative comparée à celle du Docker due à la mutualisation des run times dans le libxc. Expose plus directement les fonctionnalités du	L'application est packagée avec son run time ce qui donne une meilleure isolation au niveau applicatif. Cependant, Docker partage le noyau de l'OS hôte, donc l'isolation au niveau système reste moins stricte qu'une VM.

	noyau (cgroups, namespaces). Donc plus flexible pour contrôler la consommation de ressources par conteneur, mais moins pratique pour gérer différentes versions de runtimes applicatifs.	Pour la gestion des ressources (CPU, mémoire, I/O), Docker utilise les cgroups, ce qui limite et contrôle les ressources qu'un groupe de processus peut utiliser.
Niveau de virtualisation	Virtualisation de niveau OS	Virtualisation de niveau OS
Niveau de conteneurisation	System containers	Containers applicatifs
Tooling (API, intégration continue)	Offre surtout des outils CLI (en lignes de commande) classiques et un accès direct via SSH. L'intégration continue est donc principalement manuelle (moins de support natif pour CI/CD). Il faut gérer manuellement les dépendances entre conteneurs ou utiliser des outils externes.	Cependant, Docker dispose d'une API riche, intégrable facilement avec des systèmes CI/CD (les images Docker peuvent être construites, testées et déployées automatiquement dans des pipelines).
Performance	Plus efficace pour les tâches au niveau du système	Optimisé pour les performances de l'application, avec des conteneurs légers
Portabilité	Portabilité limitée (généralement Linux uniquement)	Très portable : fonctionne avec plusieurs OS (Windows, macOS, Linux)

3. Similitudes et différences entre les architectures d'hyperviseurs de Type 1 et de Type 2

Les hyperviseurs permettent de gérer plusieurs machines virtuelles. Il y a 2 types d'hyperviseurs:

- **Type 1 (Bare-Metal)** : L'hyperviseur s'installe directement sur le hardware (il est relié directement au matériel de la machine hôte), et il n'y a pas d'OS principal. Les machines virtuelles se partagent toutes les capacités de l'hardware. Au démarrage de la machine physique, l'hyperviseur prend le contrôle du matériel, et alloue l'intégralité des ressources aux machines hébergées.
- **Type 2 (Hosted)** : Un hyperviseur de type 2 est considéré comme un logiciel. Il s'installe et s'exécute sur un système d'exploitation déjà présent sur la machine physique. Le système d'exploitation virtualisé par un hyperviseur de type 2 (Guest OS) s'exécutera dans un troisième niveau au-dessus du matériel, celui-ci étant émulé par l'hyperviseur. L'avantage d'utiliser ce type d'hyperviseur est la

possibilité d'installer et d'exécuter autant d'hyperviseurs que l'on souhaite sur le système hôte (comme il n'est pas relié directement au matériel). Les performances sont cependant légèrement inférieures par rapport au Type 1 à cause de la couche supplémentaire.

VirtualBox → Hyperviseur de type 2 car il s'installe sur un OS hôte et dépend de celui-ci pour accéder au matériel.

OpenStack → OpenStack repose sur des hyperviseurs Type 1 (comme KVM ou Xen) pour gérer les VM dans le cloud (Source ci-dessous).

Install XenServer

Before you can run OpenStack with XenServer, you must install the hypervisor on [an appropriate server](#).

Note

Xen is a type 1 hypervisor: When your server starts, Xen is the first software that runs. Consequently, you must install XenServer before you install the operating system where you want to run OpenStack code. You then install **nova-compute** into a dedicated virtual machine on the host.

<https://docs.openstack.org/nova/pike/admin/configuration/hypervisor-xen-api.html>

Overview of KVM and Its Limitations

KVM is an open-source hypervisor that enables you to run multiple operating systems and applications on a single physical machine. It has become increasingly popular in recent years due to its ease of use and scalability.

In addition, KVM is an example of a Type-1 **hypervisor**, meaning that it runs directly on the underlying hardware rather than on a host operating system. This feature makes it more efficient than Type-2 hypervisors, which run on top of a host operating system.

<https://storware.eu/blog/kvm-vs-hyper-v-what-are-the-differences/>

TP 2 : VirtualBox VM et Docker

Dans ce TP, une machine virtuelle VirtualBox (en mode NAT) sera créée et le réseau sera configuré pour permettre une communication bidirectionnelle avec l'extérieur. L'hyperviseur VirtualBox de Type 2 en mode NAT sera utilisé pour connecter la VM au réseau. Dans un second temps, Docker sera utilisé.

A - VirtualBox

1. Création et configuration d'une VM

Pour cette première partie, nous avons commencé par copier le disque virtuel contenant le système d'exploitation Ubuntu fourni, puis nous avons décompressé l'archive afin d'obtenir le fichier image utilisable par VirtualBox.

Après avoir lancé le logiciel VirtualBox, nous avons créé une nouvelle machine virtuelle Linux / Ubuntu 64 bits. Nous lui avons attribué une mémoire vive de 1024 Mo, un processeur à un cœur et nous lui avons associé le disque dur virtuel fourni. Pour la carte réseau, nous avons choisi le mode NAT, permettant l'attribution automatique d'une adresse IP privée à la machine virtuelle.

2. Test de connectivité de la VM

La configuration suivante a été mise en place :

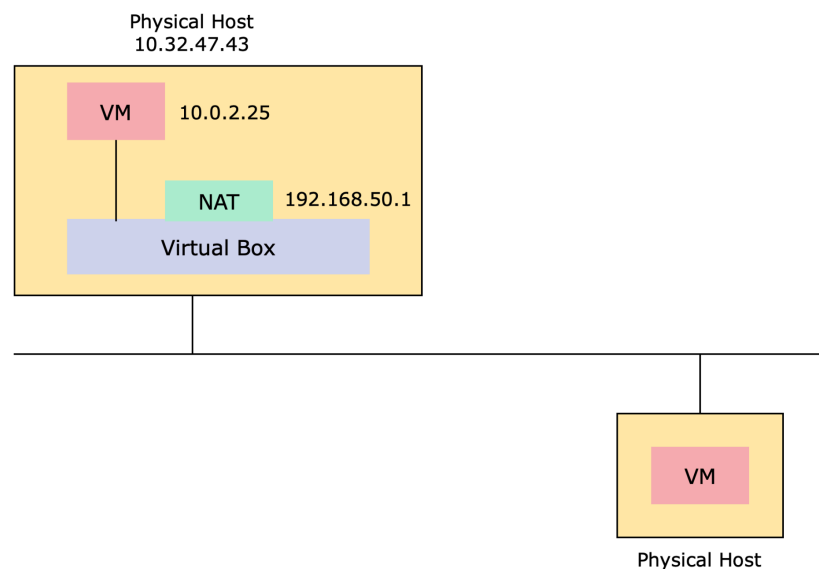


Figure 1 : Configuration mise en place.

La VM obtient une adresse privée interne fournie par VirtualBox, 10.0.2.25. L'hôte physique (Windows) possède une adresse différente, 10.32.47.43.

La VM peut accéder à Internet (ping 8.8.8.8) car VirtualBox traduit les paquets sortants de la VM via l'adresse IP de l'hôte (NAT). Par contre, il est impossible de ping la VM depuis un autre poste. Le mode NAT isole la VM du réseau local : elle n'est pas

directement accessible de l'extérieur. La VM n'est pas joignable directement depuis l'hôte non plus. L'adresse de la VM n'est en effet pas "une vraie" adresse IP dans le sens où si on ajoute une autre VM, elle aura la même adresse non routable, fournie par VirtualBox. Sans redirection de port, seule la VM initie la communication vers l'extérieur.

3. Mise en place de la connectivité (redirection de port SSH)

Dans cette partie, il s'agit de pallier la limitation observée en mode NAT pour une application dédiée (par exemple SSH sur le port 22) en utilisant la technique de redirection de port (Port Forwarding).

Pour rétablir la connectivité entrante vers une VM configurée en NAT, la méthode consiste à ajouter une règle de redirection dans l'interface de gestion de la carte réseau fournie par VirtualBox. Cette règle associe un port de la machine hôte à un port de la machine virtuelle. On choisit donc de mapper le port 1719 de l'hôte sur le port 22 de la VM.

Nom	Protocole	IP hôte	Port hôte	IP invité	Port invité
Nathalie XIV	TCP	10.1.5.87	1715	10.0.2.15	22

Figure 2 : Redirection de port (règle Nathalie XIV).

Pour les tests, on installe le serveur SSH dans la VM avec la commande `sudo apt-get install openssh-server`. Comme client SSH, on peut utiliser PuTTY sous Windows.

On exécute la commande `ssh osboxes@10.1.5.87 -p 1715` (voir Figure 3).

```
PS C:\Users\loubecjac-com> ssh osboxes@10.1.5.87 -p 1715
The authenticity of host '[10.1.5.87]:1715 ([10.1.5.87]:1715)' can't be established.
ED25519 key fingerprint is SHA256:37HAvDTLRx2EYldB+/Hrs/mL/ErFBWSo04bzeUdFmzo.
This host key is known by the following other names/addresses:
  C:\Users\loubecjac-com/.ssh/known_hosts:1: [10.1.5.87]:55555
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[10.1.5.87]:1715' (ED25519) to the list of known hosts.
osboxes@10.1.5.87's password:
Welcome to Ubuntu 22.04 LTS (GNU/Linux 5.15.0-25-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

332 updates can be applied immediately.
146 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Wed Oct  1 05:09:34 2025 from 10.0.2.2
```

Figure 3 : Résultat de la commande `ssh osboxes@10.1.5.87 -p 1715`.

Cela prouve que le port forwarding a été correctement configuré dans VirtualBox. Le port 1715 de l'hôte est redirigé vers le port 22 (SSH) de la VM. Le service OpenSSH fonctionne et la communication entre l'hôte et la VM est désormais bidirectionnelle pour cette application. La dernière ligne `Last login: ... from 10.0.2.2` indique que la connexion provient bien de l'hôte (adresse du NAT VirtualBox).

4. Duplication de la VM

La duplication du disque virtuel permet de cloner rapidement un environnement de travail. La commande fournie réalise une copie indépendante du disque. Après copie du fichier disque, on crée une nouvelle VM dans VirtualBox et on attache le fichier `disk-copy.vdi` à cette VM en choisissant le même type d'OS. Cette nouvelle VM aura la même adresse IP fournie par VirtualBox que celle créée en premier. Dupliquer une VM peut être utile pour reproduire un environnement identique (même OS, mêmes paquets installés) afin de faire des tests ou du déploiement sans repartir de zéro.

B - Docker

1. Installation du moteur Docker

L'installation commence par la mise à jour des paquets existants avec `sudo apt update`, puis l'ajout des paquets nécessaires pour permettre à `apt` de gérer les packages HTTPS. Ensuite, on ajoute la clé GPG officielle du dépôt Docker, puis on enregistre ce dépôt dans les sources d'APT. Une nouvelle mise à jour est effectuée pour prendre en compte cette source, et on vérifie que le paquet à installer (`docker-ce`) provient bien du dépôt Docker et non de celui d'Ubuntu.

L'installation se fait via `sudo apt install docker-ce`, après quoi le service Docker est automatiquement lancé et configuré pour démarrer au boot. On vérifie son bon fonctionnement avec `sudo systemctl status docker`. Cette étape garantit que le démon Docker est actif et que la commande `docker` est disponible pour manipuler les conteneurs. L'installation du moteur Docker confère un environnement complet composé du service (daemon) et du client qui communiquent via socket.

```
osboxes@osboxes:~$ sudo apt install docker-ce
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  containerd.io
Suggested packages:
  cgroupfs-mount | cgroup-lite
The following packages will be upgraded:
  containerd.io docker-ce
2 upgraded, 0 newly installed, 0 to remove and 651 not upgraded.
Need to get 51.6 MB of archives.
After this operation, 5,240 kB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 https://download.docker.com/linux/ubuntu jammy/stable amd64 containerd.io amd64 1.7.28-0-ubuntu.22.04-jammy [31.9 MB]
Get:2 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-ce amd64 5:28.4.0-1-ubuntu.22.04-jammy [19.7 MB]
Fetched 51.6 MB in 3s (16.5 MB/s)
(Reading database ... 163837 files and directories currently installed.)
Preparing to unpack .../containerd.io_1.7.28-0-ubuntu.22.04-jammy_amd64.deb ...
Unpacking containerd.io (1.7.28-0-ubuntu.22.04-jammy) over (1.6.8-1) ...
Preparing to unpack .../docker-ce_5%3a28.4.0-1-ubuntu.22.04-jammy_amd64.deb ...
Unpacking docker-ce (5:28.4.0-1-ubuntu.22.04-jammy) over (5:20.10.18-3-0-ubuntu-jammy) ...
Replacing files in old package docker-ce-cli (5:20.10.18-3-0-ubuntu-jammy) ...
Setting up containerd.io (1.7.28-0-ubuntu.22.04-jammy) ...
Setting up docker-ce (5:28.4.0-1-ubuntu.22.04-jammy) ...
Installing new version of config file /etc/default/docker ...
Installing new version of config file /etc/init.d/docker ...
Removing obsolete conffile /etc/init/docker.conf ...
Processing triggers for man-db (2.10.2-1) ...
```

Figure 4 : Résultat de la commande `sudo apt install docker-ce`.

Pour afficher un résumé complet de la configuration du client et du serveur Docker, on utilise la commande `sudo docker info`.

```
osboxes@osboxes:~$ sudo docker info
Client:
Context:    default
Debug Mode: false
Plugins:
  app: Docker App (Docker Inc., v0.9.1-beta3)
  buildx: Docker Buildx (Docker Inc., v0.9.1-docker)
  scan: Docker Scan (Docker Inc., v0.17.0)
Server:
Containers: 1
  Running: 0
  Paused: 0
  Stopped: 1
Images: 1
Server Version: 28.4.0
Storage Driver: overlay2
  Backing Filesystem: extfs
  Supports d_type: true
  Using metacopy: false
  Native Overlay Diff: true
  userxattr: false
Logging Driver: json-file
Cgroup Driver: systemd
Cgroup Version: 2
Plugins:
  Volume: local
  Network: bridge host ipvlan macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
Swarm: inactive
Runtimes: io.containerd.runc.v2 runc
Default Runtime: runc
Init Binary: docker-init
containerd version: b98a3aace656320842a23f4a392a33f46af97866
runc version: v1.3.0-0-g4ca628d1
init version: de40ad0
Security Options:
  apparmor
```

Figure 5 : Résultat de la commande `sudo docker info`.

On voit le nombre d'images et de conteneurs, la version du moteur, le type de système de fichiers utilisé (overlay2), le gestionnaire de groupes de contrôle (systemd), et les plugins disponibles. Elle permet de vérifier que Docker fonctionne correctement et que le démon est bien actif après l'installation.

2. Création et gestion de conteneurs Docker

La création de conteneurs s'effectue à partir d'images préexistantes. On commence par récupérer l'image de base d'Ubuntu avec `sudo docker pull ubuntu`.

```
osboxes@osboxes:~$ sudo docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
953cdd413371: Pull complete
Digest: sha256:353675e2a41babd526e2b837d7ec780c2a05bca0164f7ea5dbbd433d21d166fc
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest
```

Figure 6 : Résultat de la commande `sudo docker pull ubuntu`.

Ensuite, un premier conteneur (francois) est exécuté via `sudo docker run --name francois -it ubuntu`, ce qui ouvre un terminal root.

```
osboxes@osboxes:~$ sudo docker run --name francois -it ubuntu
root@ca86f8e02e71:/#
```

Figure 7 : Commande `sudo docker run --name francois -it ubuntu`.

À l'intérieur, on installe les outils nécessaires au test de connectivité (net-tools et iputils-ping). Cette configuration permet d'exécuter des commandes réseau depuis le conteneur pour observer son comportement.

```
root@ca86f8e02e71:/# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.17.0.2 netmask 255.255.0.0 broadcast 172.17.255.255
    ether a2:70:c0:bd:8d:ea txqueuelen 0 (Ethernet)
    RX packets 3062 bytes 34195478 (34.1 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2348 bytes 132926 (132.9 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Figure 8 : Résultat de la commande `ifconfig`.

L'adresse attribuée automatiquement par Docker, 172.17.0.2 (adresse privée), montre que le conteneur est connecté à un réseau virtuel privé Docker. Même si le conteneur a une IP, il ne possède pas une interface réseau physique réelle, mais une interface virtuelle reliée à la VM hôte, qui assure la communication vers l'extérieur.

Depuis le conteneur *francois*, on peut ping une ressource externe (par exemple 8.8.8.8) ce qui démontre que le conteneur sort sur Internet via la VM, cette dernière faisant office de passerelle NAT.

```
root@ca86f8e02e71:/# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=112 time=7.46 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=112 time=7.33 ms
```

Figure 9 : Ping de Google depuis le conteneur *francois*.

Le ping de la VM depuis le conteneur *francois* fonctionne également même s'ils ne sont pas sur le même réseau. En effet, le Docker Engine met en place un réseau virtuel isolé (VLAN) pour les conteneurs. Sur la machine hôte (la VM), une interface virtuelle nommée `docker0` agit comme un commutateur virtuel ou un pont (bridge). Cette interface possède l'adresse 172.17.0.1, qui sert de passerelle pour les conteneurs.

```
root@ca86f8e02e71:/# ping 10.0.2.15
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=0.066 ms
64 bytes from 10.0.2.15: icmp_seq=2 ttl=64 time=0.075 ms
64 bytes from 10.0.2.15: icmp_seq=3 ttl=64 time=0.078 ms
64 bytes from 10.0.2.15: icmp_seq=4 ttl=64 time=0.063 ms
^C
--- 10.0.2.15 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3059ms
rtt min/avg/max/mdev = 0.063/0.070/0.078/0.006 ms
```

Figure 10 : Ping de la VM depuis le conteneur *francois*.

Ainsi, quand le conteneur envoie un paquet vers la VM ou vers Internet, le NAT (Network Address Translation) configuré par le Docker Engine traduit les adresses IP internes (172.17.x.x) en adresses de la VM. C'est ce mécanisme de routage et de translation d'adresses qui permet au conteneur de communiquer avec la VM, même s'ils appartiennent à des réseaux différents.

Finalement, le ping de francois depuis la VM fonctionne également, toujours grâce au commutateur virtuel créé. Lorsque la VM envoie un ping vers l'adresse du conteneur (172.17.0.2), le paquet est routé localement via cette interface docker0, sans sortir sur le réseau physique.

```
osboxes@osboxes:~$ ping 172.17.0.2
PING 172.17.0.2 (172.17.0.2) 56(84) bytes of data.
64 bytes from 172.17.0.2: icmp_seq=1 ttl=64 time=0.063 ms
64 bytes from 172.17.0.2: icmp_seq=2 ttl=64 time=0.085 ms
64 bytes from 172.17.0.2: icmp_seq=3 ttl=64 time=0.050 ms
64 bytes from 172.17.0.2: icmp_seq=4 ttl=64 time=0.053 ms
^C
--- 172.17.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3060ms
rtt min/avg/max/mdev = 0.050/0.062/0.085/0.013 ms
```

Figure 11 : Ping de francois depuis la VM.

Ces résultats illustrent le modèle en couche de Docker : isolation des conteneurs, mais partage du noyau et de la pile réseau de l'hôte.

Un second conteneur, *celine*, est ensuite lancé avec la commande `sudo docker run --name celine -p 2223:22 -it ubuntu`. Ici, l'option `-p` redirige le port 2223 de la VM vers le port 22 du conteneur, permettant un accès SSH.

```
osboxes@osboxes:~$ sudo docker run --name celine -p 2223:22 -it ubuntu
[sudo] password for osboxes:
root@683cbb07153c:/#
```

Figure 12 : Création du conteneur celine.

On installe l'éditeur de texte Nano dans *celine* avec `apt-get -y update && apt install nano`. Une fois configuré, un snapshot de *celine* est créé. Cette opération fige l'état du conteneur en une nouvelle image réutilisable. L'avantage est de pouvoir redéployer une copie exacte du conteneur à tout moment. La commande `sudo docker ps` permet de récupérer l'identifiant des conteneurs.

```
osboxes@osboxes:~$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
683cbb07153c	ubuntu	"/bin/bash"	2 minutes ago	Up 2 minutes	0.0.0.0:2223->22/tcp, :::2223->22/tcp	celine
ca86f8e02e71	ubuntu	"/bin/bash"	12 minutes ago	Up 12 minutes		francois

Figure 13 : Résultat de la commande `sudo docker ps`.

Après suppression du conteneur *celine* (Figure 14), on liste les images disponibles (`sudo docker images`) pour vérifier la présence du snapshot (Figure 15).

```
osboxes@osboxes:~$ sudo docker rm 683cbb07153c
683cbb07153c
```

Figure 14 : Suppression de celine.

```
osboxes@osboxes:~$ sudo docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
a	version1	efc6f3a3e7a3	2 minutes ago	134MB
ubuntu	latest	6d79abd4c962	3 weeks ago	78.1MB
ubuntu	<none>	216c552ea5ba	2 years ago	77.8MB

Figure 15 : Présence du snapshot dans le répertoire "a" (version1).

Ensuite, on exécute un nouveau conteneur *CT3* basé sur cette image avec `sudo docker run --name ct3 -it a:newcont`. À l'intérieur de *CT3*, nano est toujours présent : c'est la preuve que la commande `commit` a capturé l'état complet du système de fichiers du conteneur *celine*, incluant les paquets installés. Les images Docker sont donc des couches empilées et persistantes, construites à partir des modifications effectuées sur les conteneurs.

TP 3 : OpenStack

1. Création et configuration d'une machine virtuelle sur OpenStack

Cette première partie consiste à se connecter à l'interface Web d'OpenStack à l'aide de l'identifiant INSA et à créer une VM à partir d'une image prête à l'emploi, *ubuntu4CLV*.

Lors de la première tentative de création de la machine virtuelle, une erreur est survenue. En effet, nous avons initialement essayé de l'associer au réseau public de l'INSA avec une adresse dans la plage 192.168.37.0/24. Or, n'ayant pas le droit d'utiliser une adresse publique de l'INSA, une erreur survient rapidement.

Pour corriger cette erreur, nous avons donc créé un réseau privé, avec la plage 2.14.17.0/24. Lors de la création, nous avons laissé le champ de la passerelle vide, afin que OpenStack attribue automatiquement une passerelle interne lors de la configuration du routeur.

Ensuite, nous avons recréé une nouvelle VM, nommée *fabrice*, en précisant cette fois qu'elle devait être connectée à ce réseau privé. Son adresse IP est 2.14.17.148. Nous avons ajouté une passerelle virtuelle (routeur) connectée d'un côté au réseau public (pour l'accès Internet) et de l'autre à notre réseau privé. Son adresse IP est 2.14.17.1. Cette configuration permet à la VM d'accéder à Internet via le routeur.

Voici la configuration obtenue :

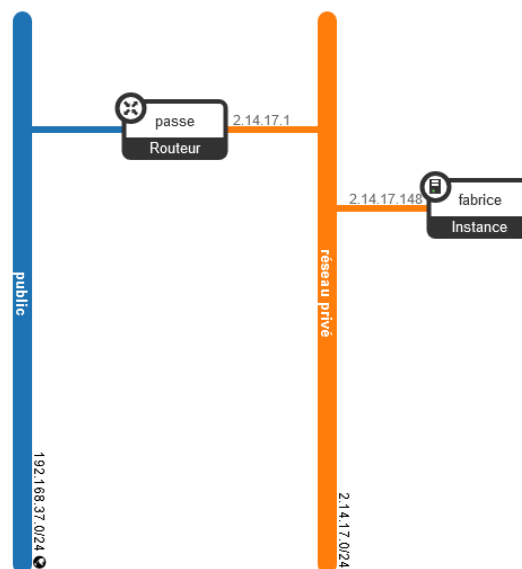


Figure 16 : Configuration obtenue après l'ajout du réseau privé et de fabrice.

Par défaut, OpenStack bloque tout le trafic réseau entrant ou sortant des VMs. Il faut donc modifier les règles de sécurité (Security Groups) pour autoriser le ping (ICMP) et les connexions SSH (port 22).

2. Tests de connectivité

La VM parvient à communiquer avec Internet ainsi qu'avec les machines du réseau interne de l'INSA. Le ping fonctionne car la VM est reliée au routeur d'OpenStack, qui fait office de passerelle vers le réseau public et assure la translation d'adresses (NAT) vers Internet ainsi que vers les machines du réseau interne.

```
user@tutorial-vm:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=113 time=8.68 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=113 time=6.91 ms
^C
--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 6.914/7.800/8.687/0.890 ms
```

Figure 17 : Ping de Google depuis la VM fabrice.

```
user@tutorial-vm:~$ ping 10.1.5.87
PING 10.1.5.87 (10.1.5.87) 56(84) bytes of data.
64 bytes from 10.1.5.87: icmp_seq=1 ttl=126 time=2.55 ms
64 bytes from 10.1.5.87: icmp_seq=2 ttl=126 time=1.09 ms
^C
--- 10.1.5.87 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 1.094/1.822/2.551/0.729 ms
```

Figure 18 : Ping d'une machine du réseau interne de l'INSA depuis la VM fabrice.

En revanche, la communication depuis une machine de l'INSA vers la VM échoue. Cette absence de connectivité s'explique par le fait que la machine externe ne connaît pas la passerelle permettant de joindre le réseau privé de la VM.

```
PS C:\Users\loubajac-com> ping 2.14.17.148

Envoi d'une requête 'Ping' 2.14.17.148 avec 32 octets de données :
Délai d'attente de la demande dépassé.
Délai d'attente de la demande dépassé.
Délai d'attente de la demande dépassé.
Délai d'attente de la demande dépassé.
```

Figure 19 : Échec Ping d'une machine du réseau interne de l'INSA depuis la VM fabrice.

Pour résoudre ce problème, nous avons alloué une adresse IP flottante. Concrètement, nous avons récupéré une adresse publique INSA (192.168.37.85) et l'avons associée à la VM *fabrice*. Dès lors, la VM dispose de deux adresses IP : une dans le réseau privé (2.14.17.148) et une autre dans le réseau public de l'INSA.

```
PS C:\Users\loubajac-com> ping 192.168.37.85

Envoi d'une requête 'Ping' 192.168.37.85 avec 32 octets de données :
Délai d'attente de la demande dépassé.
Réponse de 192.168.37.85 : octets=32 temps=2 ms TTL=62
Réponse de 192.168.37.85 : octets=32 temps=1 ms TTL=62
Réponse de 192.168.37.85 : octets=32 temps=10 ms TTL=62

Statistiques Ping pour 192.168.37.85:
    Paquets : envoyés = 4, reçus = 3, perdus = 1 (perte 25%),
    Durée approximative des boucles en millisecondes :
        Minimum = 1ms, Maximum = 10ms, Moyenne = 4ms
```

Figure 20 : Ping d'une machine du réseau interne de l'INSA depuis la VM fabrice.

L'association de l'adresse flottante a fonctionné correctement, bien que l'opération initiale ait été légèrement lente, en raison de la surcharge liée à la création et à l'association de la ressource réseau. Quand on essaye une deuxième fois de ping à la VM, l'accès s'est avéré plus rapide, probablement grâce à la mise en cache des informations.

Un test de la connectivité à l'aide d'un client SSH a été finalement effectué afin de vérifier la connectivité externe et la bonne accessibilité de la VM depuis l'extérieur du réseau privé. Ce test a pour but de s'assurer que la redirection mise en place via l'adresse flottante fonctionne correctement, et que les ports nécessaires (notamment le port 22) sont bien ouverts dans les règles de sécurité.

```
PS C:\Users\loubejac-com> ssh user@192.168.37.85
The authenticity of host '192.168.37.85 (192.168.37.85)' can't be established.
ED25519 key fingerprint is SHA256:qPxvKHgX+rqhyjpfKdTBjGVsFccDmgalVi4W10N7pK0.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.37.85' (ED25519) to the list of known hosts.
user@192.168.37.85's password:
Welcome to Ubuntu 18.04.3 LTS (GNU/Linux 4.15.0-65-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Wed Oct 1 16:06:10 CEST 2025

System load: 0.13          Processes:            163
Usage of /:  21.6% of 17.59GB Users logged in:       1
Memory usage: 68%          IP address for ens3: 2.14.17.148
Swap usage:  13%

 * Canonical Livepatch is available for installation.
   - Reduce system reboots and improve kernel security. Activate at:
     https://ubuntu.com/livepatch

477 paquets peuvent être mis à jour.
383 mises à jour de sécurité.

Nouvelle version « 20.04.6 LTS » disponible.
Lancer « do-release-upgrade » pour mettre à niveau vers celle-ci.

Last login: Fri Oct 4 01:38:33 2019 from 10.0.2.2
```

Figure 21 : Test de connectivité avec un client SSH.

Un test de la connectivité à l'aide d'un client SSH a été finalement effectué afin de vérifier la connectivité externe et la bonne accessibilité de la VM depuis l'extérieur du réseau privé. Ce test a pour but de s'assurer que la redirection mise en place via l'adresse flottante fonctionne correctement, et que les ports nécessaires (notamment le port 22) sont bien ouverts dans les règles de sécurité.

3. Redimensionnement, snapshot et restauration

On teste le redimensionnement (resize) de la VM *fabrice* en cours d'exécution. Le message d'erreur suivant est obtenu.

Danger: Une erreur s'est produite. Veuillez réessayer ultérieurement.

Figure 22 : Message d'erreur.

OpenStack ne peut pas modifier la taille d'une VM en cours d'exécution sans perturber son système de fichiers. En revanche, si elle est arrêtée, le redimensionnement est en théorie possible. Seulement, nous ne sommes pas parvenus à l'effectuer puisqu'il s'agit d'une fonctionnalité payante d'OpenStack non disponible sur le compte fournie par l'INSA.

Cela montre la souplesse de la virtualisation, qui permet d'ajuster les ressources selon les besoins. La limite technique vient du fait que OpenStack ne peut pas modifier la taille d'une VM en cours d'exécution sans perturber son système de fichiers. Une solution serait d'utiliser des technologies de live migration ou de redimensionnement dynamique de volumes.

Un snapshot est une image instantanée de la VM, incluant le système, les fichiers et les logiciels installés. Un snapshot de la VM fabrice a été effectué. Contrairement à l'image de base qui contient uniquement le système d'exploitation et les outils essentiels fournis par l'administrateur (par exemple Ubuntu), le snapshot contient tous les changements effectués : mises à jour du système, logiciels installés, configurations réseau, paquets, fichiers utilisateurs... C'est un outil essentiel pour la sauvegarde, la duplication ou la reprise après incident dans les environnements cloud.

Lors de la restauration de la VM à partir du dernier snapshot, le système retrouve exactement l'état dans lequel il se trouvait au moment de la sauvegarde. Cette opération confirme que le snapshot agit comme une copie de l'état complet de la machine, contrairement à une réinstallation depuis l'image de base. Elle permet donc de reprendre le travail sans perte de données ni de configuration

4. Topologie et spécifications de l'application Web à deux niveaux

Dans cette partie, l'objectif est de déployer une application Web sur OpenStack. Une architecture à deux niveaux (ou "2-tier") sépare la logique applicative (backend) de la présentation (frontend). Ici, le frontend correspond au point d'entrée de l'application, tandis que les différents services Node.js constituent le backend.

L'application implémente une calculatrice, dans laquelle chaque opération arithmétique (addition, soustraction, multiplication, division) est assurée par un microservice distinct. Un cinquième microservice, nommé CalculatorService, sert de point central : il reçoit les requêtes HTTP des utilisateurs, interprète les expressions et délègue les calculs aux microservices adéquats. Chaque microservice est donc exécuté indépendamment sur une VM.

Avant d'exécuter les services, nous avons installé les outils requis :

- Node.js et npm pour exécuter le code JavaScript,
- cURL pour tester les requêtes HTTP.

5. Déploiement de la calculatrice sur OpenStack

L'objectif de cette partie est de déployer l'application de calculatrice distribuée sur le cloud OpenStack, en répartissant chaque microservice sur une machine virtuelle distincte.

Six VMs ont été utilisées, chacune hébergeant un microservice. Chaque VM a été rattachée au réseau privé créé précédemment (2.14.17.0/24), assurant leur interconnexion interne. Une passerelle est utilisée pour la sortie vers Internet. Les VM utilisées sont les suivantes :

- *add* d'adresse IP 2.14.17.55 (pour l'addition),
- *mule* d'adresse IP 2.14.17.139 (pour la multiplication),
- *div* d'adresse IP 2.14.17.114 (pour la division),
- *sous* d'adresse IP 2.14.17.113 (pour la soustraction),
- *fabrice* d'adresse IP 2.14.17.148 (hébergeant CalculatorService),
- *solangelacliente* d'adresse IP 2.14.17.103 (servant de client HTTP).

La Figure 23 présente la configuration obtenue.

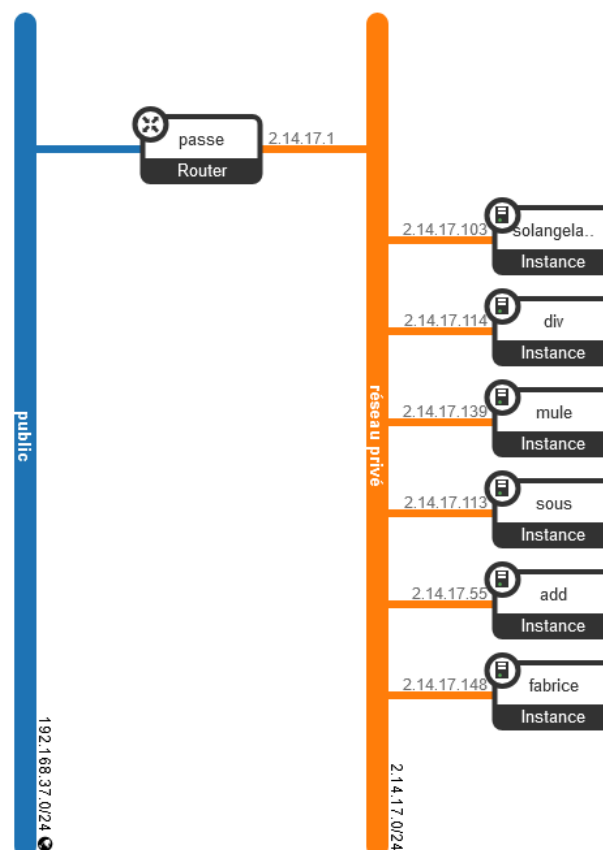
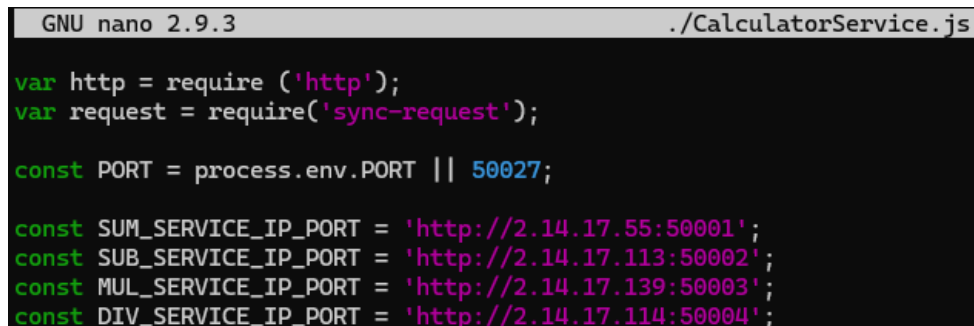


Figure 23 : Configuration des VM.

Nous avons modifié le fichier `CalculatorService.js` afin d'y renseigner les adresses IP et ports des différents microservices déployés chacun sur une VM distincte.

Chaque VM héberge donc un service `Node.js` écoutant sur un port spécifique (de 50001 à 50004). Le service principal, `CalculatorService`, fait office d'orchestrateur : il reçoit les requêtes de calculs et les redirige vers les microservices concernés, en fonction de l'opération à effectuer.



```
GNU nano 2.9.3 ./CalculatorService.js

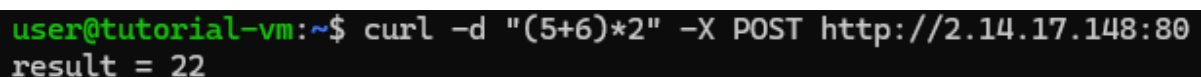
var http = require('http');
var request = require('sync-request');

const PORT = process.env.PORT || 50027;

const SUM_SERVICE_IP_PORT = 'http://2.14.17.55:50001';
const SUB_SERVICE_IP_PORT = 'http://2.14.17.113:50002';
const MUL_SERVICE_IP_PORT = 'http://2.14.17.139:50003';
const DIV_SERVICE_IP_PORT = 'http://2.14.17.114:50004';
```

Figure 24 : Modification du fichier `CalculatorService.js`.

Depuis la VM cliente *solangelacliente*, nous avons exécuté la commande `curl -d "(5+6)*2" -X POST http://2.14.17.148:80`, pour tester le bon fonctionnement du service. Le port 80 correspond en effet au port par défaut du protocole HTTP sur lequel écoute la VM serveur *fabrice*.



```
user@tutorial-vm:~$ curl -d "(5+6)*2" -X POST http://2.14.17.148:80
result = 22
```

Figure 25 : Test de fonctionnement.

6. Déploiement et configuration d'une topologie réseau multi-niveaux sur OpenStack

L'objectif de cette partie est de mettre en place une application web (la calculatrice) répartie sur deux sous-réseaux, avec une séparation claire entre le front-end et les services de calcul. Le réseau doit permettre à la fois un accès sécurisé pour les utilisateurs et une communication interne entre les différents services applicatifs. On joue le rôle d'un fournisseur de service de réseau (NaaS).

Nous avons commencé par déployer deux sous-réseaux distincts. Le premier, *nouveau réseau privé jiji*, d'adresse IP 4.7.9.0/24, héberge le `CalculatorService` (front-end) ainsi que les routeurs assurant la liaison vers le réseau public. Le second, nommé *réseau privé*, (2.14.17.0/24) regroupe les quatre microservices réalisant les opérations. Après avoir créé ces réseaux dans OpenStack, nous avons instancié les différentes machines virtuelles et déployé sur chacune le service correspondant. La topologie du réseau est présentée à la Figure 26. La VM *fabricelebon* héberge le service `CalculatorService`. Le routeur *titi* fait le lien entre les deux sous-réseaux privés créés. Depuis le réseau public les paquets empruntent la passerelle *passee* entre *public* et *nouveau réseau privé jiji*, puis passent sur le réseau *réseau privé* à l'aide du routeur *titi*.

Lors des premiers tests, la connectivité entre le `CalculatorService` et les VMs du réseau 2.14.17.0/24 n'était pas fonctionnelle. Le problème provenait de l'absence de règle de routage entre les deux sous-réseaux : les paquets émis par la VM front-end n'avaient

pas de chemin défini vers les adresses du réseau *réseau privé*. En analysant la table de routage, nous avons constaté que la passerelle par défaut du CalculatorService n'incluait pas de route vers 2.14.17.0/24.

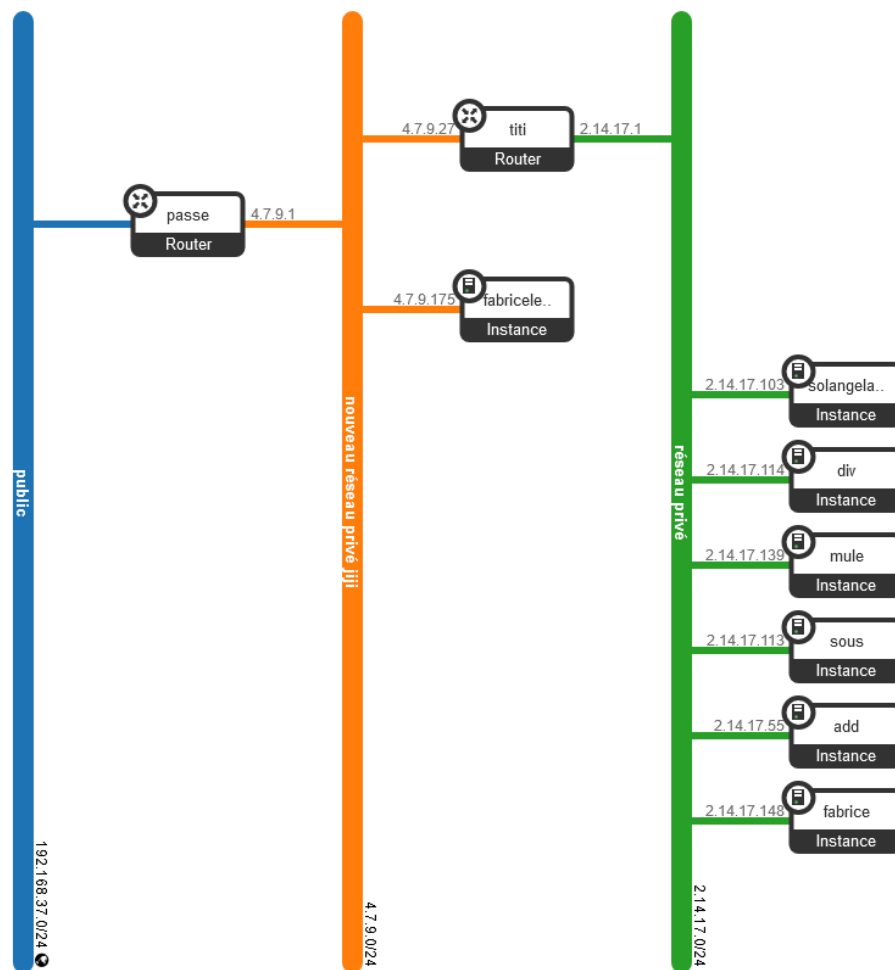


Figure 26 : Topologie du réseau obtenue.

Pour corriger cette situation, nous avons ajouté une route statique vers le réseau 2.14.17.0/24 en spécifiant comme passerelle l'adresse du routeur *titi*. Concrètement, la commande suivante a été exécutée (Figure 27).

```
user@tutorial-vm:~$ sudo route add -net 2.14.17.0/24 gw 4.7.9.27/24
```

Figure 27 : Route statique ajoutée.

La communication devient alors complète : un utilisateur envoie une requête depuis Internet, celle-ci passe d'abord par la passerelle publique vers le réseau *nouveau réseau privé jiji*, puis traverse le routeur *titi*, avant d'atteindre *réseau privé* pour effectuer le calcul. Inversement, les réponses ou les requêtes internes émises par CalculatorService (*fabricelebon*) vers les services de calcul empruntent le chemin inverse (Figure 28).

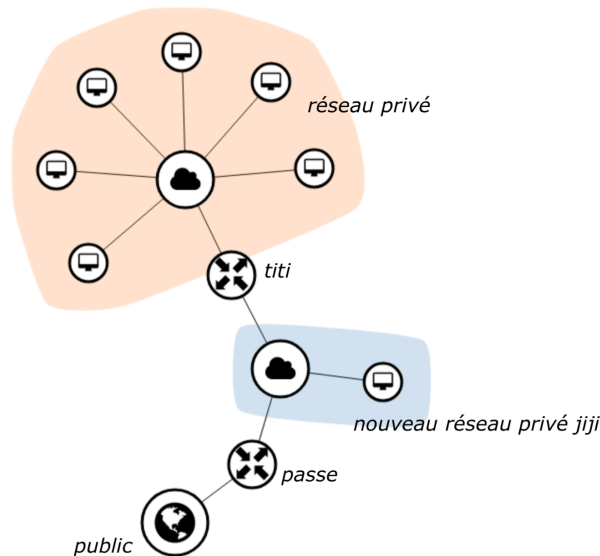


Figure 28 : Topologie du réseau obtenue et chemin des paquets.

Après la mise en place du routage, les tests de connectivité (ping, curl) entre CalculatorService et les quatre services arithmétiques ont été concluants.

```
U:\>curl -d "((5+6)*(16-2))/4" -X POST http://192.168.37.70:50027
result = 38.5
```

Figure 29 : Test des services arithmétiques.

7. Automatisation du déploiement de la topologie

Dans cette partie, on va se consacrer à automatiser le déploiement de la topologie, pour cela nous allons écrire un script Python. Cette automatisation permet de simplifier le déploiement des topologies et de facilement déployer des installations.

Dans un premier temps nous allons installer l'environnement Python :

1. On va installer Python et pip qui nous sera utile pour installer un module (Figure 30)

```
jp@Norbert:~$ sudo apt install python3 python3-pip
```

Figure 30 : Installation de Python et pip.

2. Maintenant que Python et pip sont installés, on peut installer la librairie Python `openstacksdk` qui permet de communiquer avec l'API d'Openstack. Voici la commande à exécuter en utilisant pip (Figure 31).

```
jp@Norbert:~$ pip install openstacksdk
```

Figure 31 : Installation d'openstacksdk.

L'environnement Python installé, il nous faut toutes les informations qui permettent à notre script de se connecter à notre compte Openstack. Pour cela, il faut récupérer le fichier *openrc.sh*. Une des façons de le récupérer est de se connecter à l'application Web d'Openstack et de cliquer sur son nom puis de cliquer sur "Openstack RC file" (Figure 32).

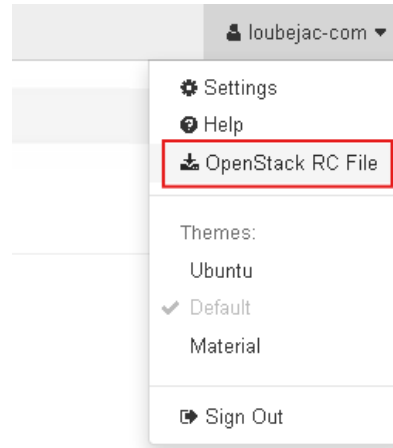


Figure 32 : Téléchargement du fichier *openrc*.

Avec le fichier *openrc.sh*, on peut maintenant ajouter ces informations dans les variables d'environnement. Pour cela, on utilise la commande `source` (Figure 33).

```
osboxes@osboxes:~/Downloads$ source ./5ISS-A1-5-openrc.sh
Please enter your OpenStack Password for project 5ISS-A1-5 as user loubejac-com:
```

Figure 33 : Ajout des variables d'environnement nécessaires.

La préparation de l'environnement est finie, on peut passer à la programmation du script.

Le script complet est disponible sur un répertoire public sur github (<https://github.com/jploubejac/INSA-5ISS-TP-CEC-python-code>).

Nous avons décidé de séparer le script en deux fonctions, une pour la création des réseaux et une pour la création des machines.

Nous avons aussi créé une fonction pour supprimer les réseaux et les machines, pour nous permettre de tester le script plus facilement.

La fonction "réseaux" (Figure 34), prend en argument la connexion à Openstack et permet de récupérer le subnet public, et créer deux réseaux ; avec pour le premier, deux sous réseaux et pour le second qu'un seul sous réseau. La fonction crée aussi les 2 routeurs. L'un entre le premier réseau privé et le réseau public et l'autre entre les deux réseaux privés. La fonction renvoie les réseaux privés créés.

```
def réseaux(conn):
    public_network = conn.network.find_network("public")
    sub_public = None
    for subnet in conn.network.subnets(network_id=public_network.id):
        if subnet.name == "sub-public":
            sub_public = subnet
            break
    if sub_public is None: return errprint("Public subnet not found")
    network1 = conn.network.create_network(name="mynetwork1")
    print("Le réseau Network1 ID:", network1.id)
    subnet1a = conn.network.create_subnet(
        name="mysubnet1a",
        network_id=network1.id,
        ip_version='4',
        cidr='017.015.014.0/25'
    )
    print("Le sous-réseau Subnet1a ID:", subnet1a.id)
    subnet1b = conn.network.create_subnet(
        name="mysubnet1b",
        network_id=network1.id,
        ip_version='4',
        cidr='017.015.014.128/25'
    )
    print("Le sous-réseau Subnet1b ID:", subnet1b.id)
    network2 = conn.network.create_network(name="mynetwork2")
    print("Le réseau Network2 ID:", network2.id)
    subnet2 = conn.network.create_subnet(
        name="mysubnet2",
        network_id=network2.id,
        ip_version='4',
        cidr='016.038.014.0/24'
    )
    print("Le sous-réseau Subnet2 ID:", subnet2.id)
    router_public_1 = conn.network.create_router(name="router_public_1")
    print("Le routeur Router_Public_1 ID:", router_public_1.id)
    conn.network.add_interface_to_router(router_public_1, subnet_id=subnet1a.id)
    conn.network.update_router(router_public_1, external_gateway_info={"network_id": public_network.id})
    print("Le routeur Router_Public_1 a été connecté au sous-réseau Subnet1a avec comme passerelle (gateway) le réseau public.")

    router_1_2 = conn.network.create_router(name="router_1_2")
    print("Le routeur Router_1_2 ID:", router_1_2.id)
    conn.network.add_interface_to_router(router_1_2, subnet_id=subnet1b.id)
    conn.network.add_interface_to_router(router_1_2, subnet_id=subnet2.id)
    print("Le routeur Router_1_2 a été connecté au sous-réseau Subnet1b et au sous-réseau Subnet2.")
    print("Configuration des réseaux terminée")
    return network1, network2
```

Figure 34 : Fonction réseaux.

La fonction "machine" (Figure 35), prend en argument la connexion Openstack ainsi que les réseaux privés. Elle crée les machines virtuelles, en utilisant l'image "Ubuntu4CLV" et le flavor "small2".

```
def machine(conn, network1, network2):
    image = conn.compute.find_image("Ubuntu4CLV")
    flavor = conn.compute.find_flavor("small2")
    name = ["fabricelebon_py", "solangelacclient_py", "div_py", "mule_py", "sous_py", "add_py"]

    for n in name:
        if n == "fabricelebon_py":
            net = network1.id
        else:
            net = network2.id
        server = conn.compute.create_server(
            name=n,
            image_id=image.id,
            flavor_id=flavor.id,
            networks=[{"uuid": net}]
        )
        print("La machine", n, "a été créée avec l'ID:", server.id)
    print("Toutes les machines ont été créées")
```

Figure 35 : Fonction machine.

La fonction "nettoyage" (Figure 36), prend en argument la connexion Openstack. La fonction supprime les machines, les routeurs, les réseaux et sous réseaux, créés par le script.

```
def nettoyage(conn):
    for server in conn.compute.servers():
        if server.name in ["fabricelebon_py", "solangelacclient_py", "div_py", "mule_py", "sous_py", "add_py"]:
            print(f"Suppression de la machine {server.name} ({server.id})...")
            conn.compute.delete_server(server, ignore_missing=True)
            conn.compute.wait_for_delete(server)

    for router in conn.network.routers():
        if router.name in ["router_public_1", "router_1_2"]:
            print(f"Suppression du routeur {router.name} ({router.id})...")
            try:
                conn.network.update_router(router, external_gateway_info=None)
                print(" Gateway supprimée.")
            except Exception as e:
                print(f" Erreur lors de la suppression de la gateway : {e}")
            ports = list(conn.network.ports(device_id=router.id))
            for port in ports:
                for ip in port.fixed_ips:
                    try:
                        conn.network.remove_interface_from_router(router, subnet_id=ip['subnet_id'])
                        print(f" Interface sur subnet {ip['subnet_id']} détachée.")
                    except Exception as e:
                        print(f" Erreur lors du détachement de l'interface : {e}")
            try:
                conn.network.delete_router(router, ignore_missing=True)
                print(f" Routeur supprimé.")
            except Exception as e:
                print(f" Erreur lors de la suppression du routeur : {e}")

    for subnet in conn.network.subnets():
        if subnet.name in ["mysubnet1", "mysubnet2"]:
            print(f"Suppression du sous-réseau {subnet.name} ({subnet.id})...")
            conn.network.delete_subnet(subnet, ignore_missing=True)

    for net in conn.network.networks():
        if net.name in ["mynetwork1", "mynetwork2"]:
            print(f"Suppression du réseau {net.name} ({net.id})...")
            conn.network.delete_network(net, ignore_missing=True)

    print("Nettoyage terminé")
```

Figure 36 : Fonction nettoyage.

Après l'exécution du script, on obtient la topologie suivante, identique à celle obtenue précédemment :

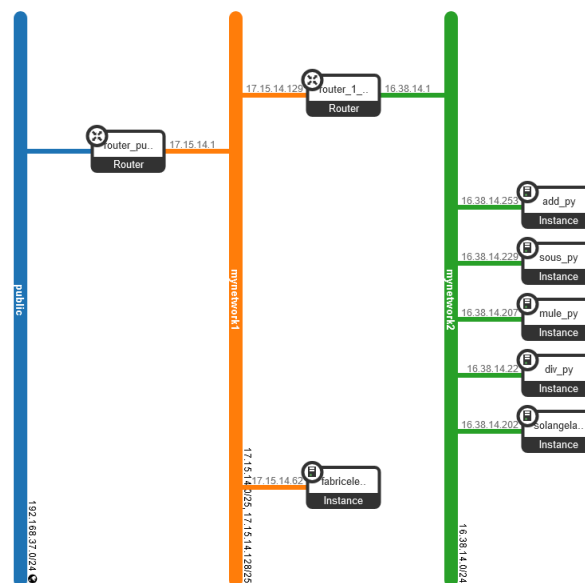


Figure 37 : Topologie obtenue après l'exécution du script.

TP 4 : Orchestration de services dans un environnement hybride cloud/edge

Ce TP porte sur la mise en œuvre d'une infrastructure d'edge computing destinée à illustrer la gestion de services à faible latence à l'aide de Kubernetes. L'objectif est d'étudier le cas des voitures autonomes, dont les modules de décision dépendent de traitements distribués sur des nœuds proches pour réduire la latence. L'infrastructure repose sur un cluster Kubernetes composé de trois machines virtuelles : un nœud master, chargé de la gestion du cluster (réseau, stockage, conteneurs), et deux workers, dédiés à l'exécution des services. L'infrastructure controller communique avec le master pour déployer dynamiquement les services sur les nœuds edge. Le TP comporte trois parties : la mise en place du cluster, le déploiement et l'exécution des services, puis la migration et la gestion des services en temps réel. Seule la première partie a été traitée.

1. Configuration de l'infrastructure Cloud

La première étape de ce TP consiste à mettre en place une infrastructure cloud fonctionnelle reposant sur un cluster Kubernetes à trois nœuds.

L'ensemble des machines virtuelles a été créé sur la plateforme OpenStack de l'INSA. Après authentification via l'interface web, un réseau privé a été créé puis relié au réseau public avec une passerelle. Trois instances Ubuntu 20 ont ensuite été déployées : une instance de type medium pour le master et deux de type small2 pour les workers. Chaque machine s'est vu attribuer une adresse IP flottante pour permettre les connexions SSH depuis la machine locale. Le tableau suivant montre les différentes instances créées.

	Nom de l'instance	Adresse IP flottante	Adresse IP privée
Master	<i>maitresse_charline</i>	192.168.37.70	27.9.27.39
Worker 1	<i>ouvrier_guy</i>	192.168.37.28	27.9.27.68
Worker 2	<i>ouvriere_ginette</i>	192.168.37.68	27.9.27.113

Sur chaque instance, un utilisateur "user" a été créé, ajouté au groupe sudo et autorisé à se connecter par mot de passe via la configuration du service SSH. Après redémarrage du service, la connexion simultanée aux trois serveurs a permis de préparer le cluster.

Le swap a été désactivé pour assurer le bon fonctionnement de kubelet, et chaque machine a reçu un nom d'hôte distinct (identique au nom de l'instance). Les paquets ont ensuite été mis à jour, puis les composants essentiels de Kubernetes (kubeadm, kubelet, kubectl) installés depuis le dépôt officiel, avant d'être figés pour éviter toute mise à jour involontaire.

```
user@ubuntu:~$ sudo swapoff -a
[sudo] password for user:
user@ubuntu:~$ sudo hostnamectl set-hostname ouvriere_ginette
```

Figure 38 : Commande `swapoff -a`.

```
user@ubuntu:~$ sudo hostnamectl status
Static hostname: ouvrieregnette
Pretty hostname: ouvriere_ginette
Icon name: computer-vm
Chassis: vm
Machine ID: 220fd060492ff943c75b5675c20b46a7
Boot ID: 55bf3689eb8e405c9c1cb47979d18bb5
Virtualization: kvm
Operating System: Ubuntu 20.04.6 LTS
Kernel: Linux 5.4.0-163-generic
Architecture: x86-64
```

Figure 39 : Commande `sudo hostnamectl status`.

Par la suite, Docker a été installé et configuré avec le pilote systemd afin d'assurer la compatibilité avec Kubernetes. Après redémarrage du service, la configuration du noyau a été ajustée pour activer le routage IP et le filtrage des paquets nécessaires à la communication entre les conteneurs. Le fichier de configuration de kubelet a été modifié sur chaque machine afin d'y renseigner l'adresse IP privée de l'interface ens3, garantissant la cohérence du réseau interne du cluster.

Le cluster a ensuite été initialisé depuis le nœud master (*maitresse_charline*) à l'aide de la commande `kubeadm init`. Après création du fichier de configuration kubeconfig, les deux workers ont rejoint le cluster grâce au token généré lors de l'initialisation. Une fois l'opération terminée, la commande `kubectl get nodes` a permis de vérifier l'état des nœuds : ceux-ci apparaissent dans un état "NotReady".

```
user@maitressecharline:/usr/lib/systemd/system/kubelet.service.d$ kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
maitressecharline   NotReady  control-plane  16m   v1.28.1
ouvrieregnette      NotReady  <none>       7m27s v1.28.1
ouvrierguy          NotReady  <none>       7m22s v1.28.1
```

Figure 40 : Commande `kubectl get nodes`, nœuds "Not Ready".

Les nœuds ne sont pas encore prêts à exécuter des pods, car Kubernetes ne dispose pas nativement de solution de gestion réseau.

Pour remédier à cela, le plugin Calico a été déployé. Une fois appliqué, le cluster passe à l'état "Ready" : les trois nœuds communiquent correctement. La configuration du réseau overlay est fonctionnelle. L'infrastructure de base devient donc prête à accueillir les services cloud et edge.

```
user@maitressecharline:/usr/lib/systemd/system/kubelet.service.d$ kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
maitressecharline   Ready     control-plane  16m   v1.28.1
ouvrieregnette      Ready     <none>       7m27s v1.28.1
ouvrierguy          Ready     <none>       7m22s v1.28.1
```

Figure 41 : Commande `kubectl get nodes`, nœuds "Ready".

```
user@maitressecharline: /usr/lib/systemd/system/kubelet.service.d$ kubectl get pods -A
NAMESPACE NAME READY STATUS RESTARTS AGE
kube-system calico-kube-controllers-658d97c59c-mf8cm 1/1 Running 0 110s
kube-system calico-node-68crr 1/1 Running 0 110s
kube-system calico-node-jpv8v 1/1 Running 0 110s
kube-system calico-node-wzb22 1/1 Running 0 110s
kube-system coredns-5dd5756b68-2nfj9 1/1 Running 0 16m
kube-system coredns-5dd5756b68-gx2qv 1/1 Running 0 16m
kube-system etcd-maitressecharline 1/1 Running 0 16m
kube-system kube-apiserver-maitressecharline 1/1 Running 0 16m
kube-system kube-controller-manager-maitressecharline 1/1 Running 0 16m
kube-system kube-proxy-ds4sz 1/1 Running 0 7m59s
kube-system kube-proxy-kv8tc 1/1 Running 0 16m
kube-system kube-proxy-x4qd9 1/1 Running 0 7m54s
kube-system kube-scheduler-maitressecharline 1/1 Running 0 16m
user@maitressecharline: /usr/lib/systemd/system/kubelet.service.d$ kubectl get nodes -o wide
NAME STATUS ROLES AGE VERSION INTERNAL-IP EXTERNAL-IP OS-IMAGE KERNEL-VERSION CONTAINER-RUNTIME
maitressecharline Ready control-plane 17m v1.28.1 27.9.27.39 <none> Ubuntu 20.04.6 LTS 5.4.0-163-generic containerd://1.7.24
ouvrieregnette Ready <none> 8m50s v1.28.1 27.9.27.113 <none> Ubuntu 20.04.6 LTS 5.4.0-163-generic containerd://1.7.24
ouvrierguy Ready <none> 8m45s v1.28.1 27.9.27.68 <none> Ubuntu 20.04.6 LTS 5.4.0-163-generic containerd://1.7.24
```

Figure 42 : Commandes `kubectl get pods -A` et `kubectl get nodes -o wide`, nœuds "Ready".

Kubernetes agit comme un orchestrateur de conteneurs, chargé d'assurer le déploiement, la gestion et la résilience des services. Les conteneurs sont encapsulés dans des objets appelés *pods*, unités minimales d'exécution dans le cluster. Pour garantir la persistance des applications, on peut créer plusieurs copies de ces pods (à l'aide d'objets *deployments*). Cela assure leur redémarrage automatique en cas de défaillance.

Afin de fournir un accès réseau stable aux pods, Kubernetes définit des services qui font office d'intermédiaires. Trois types de services existent : ClusterIP, accessible uniquement à l'intérieur du cluster, NodePort, exposant les applications sur le réseau des différents nœuds, et LoadBalancer, qui permet un accès externe via un équilibreur de charge. Dans cette partie, seuls les deux premiers ont été étudiés.

Une application simple a été déployée pour illustrer leur fonctionnement. Après création d'un dépôt Docker Hub et configuration d'un secret Kubernetes permettant au cluster d'y accéder, les nœuds Worker1 et Worker2 ont été labellisés respectivement PoP=space_1 et PoP=space_2 afin de simuler deux zones géographiques distinctes.

```
user@maitressecharline: /usr/lib/systemd/system/kubelet.service.d$ kubectl label node ouvrierguy PoP=space_1
node/ouvrierguy labeled
user@maitressecharline: /usr/lib/systemd/system/kubelet.service.d$ kubectl label node ouvrieregnette PoP=space_2
node/ouvrieregnette labeled
```

Figure 43 : Labellisation des workers *ouvrierguy* et *ouvrieregnette*.

Le dépôt `kube_service` a été cloné sur le nœud master *maitresse_charline*, puis l'application a été déployée avec le service ClusterIP.

```
user@maitressecharline:~/kube_service/ClusterIP$ cat app_deployment_1.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: fastapi-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: fastapi-app
  template:
    metadata:
      labels:
        app: fastapi-app
    spec:
      imagePullSecrets:
        - name: repo-key
      containers:
        - name: fastapi-app-container
          image: yxos/fastapi-app
          imagePullPolicy: Always
          ports:
            - containerPort: 5000
              protocol: TCP
      nodeSelector:
        PoP: space_1
```

Figure 44 : Déploiement de l'application.

La commande `kubectl get pods -o wide` a permis de vérifier la répartition des pods sur les nœuds. On observe que trois pods *fastapi-app* ont été créés et qu'ils sont tous en état Running, avec chacun une adresse IP distincte.

```
user@maitressecharline:~/kube_service/ClusterIP$ kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
fastapi-app-76f6674775-l867g        1/1     Running   0           2m54s  192.168.221.131 ouvrierguy       <none>            <none>
fastapi-app-76f6674775-t4dmn        1/1     Running   0           2m54s  192.168.221.130 ouvrierguy       <none>            <none>
fastapi-app-76f6674775-tlx9w        1/1     Running   0           2m54s  192.168.221.129 ouvrierguy       <none>            <none>
```

Figure 45 : Commande `kubectl get pods -o wide`.

On remarque que les trois pods ont été déployés sur un seul nœud (*ouvrierguy*). Cela montre la politique d'ordonnancement par défaut de Kubernetes : en l'absence de contraintes, les pods sont placés sur le nœud le plus disponible.

Pour observer les services disponibles, la commande `kubectl get services -o wide` a permis d'afficher le service associé à l'application FastAPI. Ensuite, à l'aide de la commande `kubectl describe services`, on remarque que la ligne Endpoints liste les adresses IP internes des pods déployés. Ces adresses appartiennent au réseau virtuel interne de Kubernetes et ne sont donc accessibles qu'à l'intérieur du cluster.

```
user@maitressecharline:~$ kubectl get services -o wide
NAME                                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE   SELECTOR
fastapi-app-clusterip-service       ClusterIP    10.108.32.7   <none>         80/TCP     6m38s  app=fastapi-app
kubernetes                           ClusterIP    10.96.0.1     <none>         443/TCP    37m    <none>

user@maitressecharline:~$ kubectl describe services fastapi-app-clusterip-service
Name:                                fastapi-app-clusterip-service
Namespace:                           default
Labels:                               <none>
Annotations:                         <none>
Selector:                             app=fastapi-app
Type:                                 ClusterIP
IP Family Policy:                     SingleStack
IP Families:                          IPv4
IP:                                   10.108.32.7
IPs:                                  10.108.32.7
Port:                                 <unset> 80/TCP
TargetPort:                           5000/TCP
Endpoints:                            192.168.221.129:5000,192.168.221.130:5000,192.168.221.131:5000
Session Affinity:                     None
Events:                               <none>
```

Figure 46 : Services déployés et leurs descriptions.

Lorsqu'on tente d'accéder à l'un de ces endpoints depuis le nœud master avec la commande `curl`, aucune réponse n'est renvoyée. Cela s'explique par le fait qu'un service de type ClusterIP n'est visible que depuis l'intérieur du cluster Kubernetes. Le master ne faisant pas partie du réseau de pods, il ne peut joindre directement les IP internes associées aux services. ClusterIP fournit donc une isolation réseau stricte.

Pour vérifier le comportement interne, un pod temporaire a été créé à l'aide de la commande `kubectl run testpod --image=nginx`.

```
{"hello":"world"}user@maitressecharline:~$ kubectl run testpod --image=nginx
pod/testpod created
```

Figure 47 : Création de `testpod`.

Après avoir listé les pods avec `kubectl get pods`, le `testpod` a été ouvert en mode interactif avec `kubectl exec --stdin --tty testpod -- /bin/bash`. Depuis ce pod, une nouvelle requête `curl` a été exécutée.

```
user@maitressecharline:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
fastapi-app-76f6674775-l867g       1/1     Running   0           18m
fastapi-app-76f6674775-t4dmn       1/1     Running   0           18m
fastapi-app-76f6674775-tlx9w       1/1     Running   0           18m
testpod                             0/1     ContainerCreating   0           12s
```

Figure 48 : Commande `kubectl get pods`.

```
user@maitressecharline:~$ kubectl exec --stdin --tty testpod -- /bin/bash
root@testpod:/# curl http://192.168.221.129:5000
{"hello":"world"}root@testpod:/# ^C
```

Figure 49 : Commande `curl` réussie !

Cette fois, la requête renvoie une réponse HTTP valide. Le contenu affiché correspond à la réponse par défaut de l'application. Le service est donc fonctionnel et accessible à l'intérieur du réseau interne du cluster.

On souhaite maintenant déployer une application en utilisant le service NodePort. Pour cela, on commence par supprimer les ressources déjà déployées. La commande `kubectl apply -f ./kube_service/NodePort` a été exécutée sur le nœud master. Cette étape déploie l'application FastAPI avec un service NodePort, qui ouvre un port spécifique sur chaque nœud du cluster. Pour vérifier la bonne exécution du déploiement, on a utilisé la commande `kubectl get pods -o wide` (Figure 50).

```
user@maitressecharline:~$ kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
fastapi-app-76f6674775-dc9jf        1/1     Running   0           41s   192.168.221.134 ouvrierguy       <none>            <none>
fastapi-app-76f6674775-m185p        1/1     Running   0           41s   192.168.221.133 ouvrierguy       <none>            <none>
fastapi-app-76f6674775-vbbgl        1/1     Running   0           41s   192.168.221.135 ouvrierguy       <none>            <none>
testpod                             1/1     Running   0           8m35s 192.168.221.132 ouvrierguy       <none>            <none>
```

Figure 50 : Commande `get pods -o wide`.

Contrairement au service ClusterIP, NodePort expose l'application sur un port spécifique de chaque nœud du cluster. La commande `kubectl get services -o wide` montre ce port, il s'agit du port 31331.

```
user@maitressecharline:~$ kubectl describe services fastapi-app-service
Name: fastapi-app-service
Namespace: default
Labels: <none>
Annotations: <none>
Selector: app=fastapi-app
Type: NodePort
IP Family Policy: SingleStack
IP Families: IPv4
IP: 10.104.16.137
IPs: 10.104.16.137
Port: <unset> 80/TCP
TargetPort: 5000/TCP
NodePort: <unset> 31331/TCP
Endpoints: 192.168.221.133:5000,192.168.221.134:5000,192.168.221.135:5000
Session Affinity: None
External Traffic Policy: Cluster
Events: <none>
user@maitressecharline:~$
```

Figure 51 : Commande `kubectl get services -o wide`.

En comparant avec le fichier YAML du service, on remarque que ce port n'est pas explicitement mentionné : Kubernetes l'a attribué dynamiquement dans la plage 30000–32767.

```
user@maitressecharline:~/kube_service/ClusterIP$ cat app_service_cluster_ip.yaml
apiVersion: v1
kind: Service
metadata:
  name: fastapi-app-clusterip-service
spec:
  selector:
    app: fastapi-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5000
  type: ClusterIP
user@maitressecharline:~/kube_service/ClusterIP$
```

Figure 52 : Contenu du fichier YAML.

Enfin, on lance depuis le master la commande `curl`. La requête retourne la réponse du service FastAPI.

```
user@maitressecharline:/usr/lib/systemd/system/kubelet.service.d $ curl http://192.168.221.133:31331
{"hello":"world"}
user@Masternode:/usr/lib/systemd/system/kubelet.service.d $ curl http://192.168.221.133:31331
{"hello":"world"}
user@maitressecharline:/usr/lib/systemd/system/kubelet.service.d $ curl http://192.168.221.134:31331
{"hello":"world"}
user@Masternode:/usr/lib/systemd/system/kubelet.service.d $ curl http://192.168.235.134:31331
{"hello":"world"}
user@maitressecharline:/usr/lib/systemd/system/kubelet.service.d $ curl http://192.168.235.135:31331
{"hello":"world"}
user@maitressecharline:/usr/lib/systemd/system/kubelet.service.d $ ^C
```

Figure 53 : Commande `curl`.

Cette fois, l'accès est possible depuis l'extérieur du cluster. Le service NodePort agit comme un pont entre le réseau interne Kubernetes et le réseau des autres nœuds. Les requêtes reçues sur ce port sont redirigées automatiquement vers les pods en cours d'exécution. Cela valide le fonctionnement du routage externe : Kubernetes est capable d'exposer des services aux utilisateurs finaux.

Finalement, toutes les ressources ont ensuite été supprimées.

```
user@maitressecharline:~/kube_service$ kubectl delete -f ./NodePort
deployment.apps "fastapi-app" deleted
ingress.networking.k8s.io "fastapi-app-ingress" deleted
service "fastapi-app-service" deleted
```

Figure 54 : Suppression des ressources.

Conclusion

Les travaux pratiques de ce module ont permis d'étudier les différentes couches de la virtualisation et du cloud computing, depuis les hyperviseurs jusqu'à l'orchestration de services. Nous avons d'abord compris les différences fondamentales entre machines virtuelles et conteneurs, puis expérimenté leur mise en œuvre à travers VirtualBox et Docker. L'utilisation d'OpenStack a ensuite illustré la création et la gestion d'une infrastructure cloud complète, avec la mise en place d'une application distribuée et la configuration réseau associée. Enfin, la découverte de Kubernetes a introduit la logique d'orchestration et de déploiement automatisé dans un environnement hybride cloud/edge. Les manipulations de ces TP nous ont permis de comprendre les services virtualisés, de leur isolation à leur interconnexion.

